Instituto Politécnico Nacional

Escuela Superior de Cómputo

Algoritmos Bioinspirados

Práctica 2.
Algoritmo genético con codificación entera para permutaciones

Bustillos Cruz Jonatan

Gutiérrez Sánchez Angélica

Grupo 5BM2

Índice

1. Introducción **4**
1.1. Objetivo . . . 4
1.2. Organización del documento . . . 4

2. Marco Teórico **5**
2.1. Algoritmos genéticos . . . 5
2.2. Juego del Sudoku . . . 5
2.3. Gráficas de convergencia . . . 5

3. Implementación de algoritmo genético con codificación entera **6**
3.1. Selección de padres . . . 6
3.1.1. Torneo binario determinista . . . 6
3.2. Recombinación . . . 6
3.3. Mutación . . . 7
3.3.1. Mutación por intercambio y por reinicialización . . . 7
3.4. Búsqueda local . . . 8
3.4.1. Búsqueda local por columna y por sub-bloque . . . 8
3.5. Diagrama de flujo . . . 8

4. Pruebas y resultados **10**
4.1. Metodología de evaluación . . . 10
4.2. Comparativa general por niveles de dificultad . . . 10
4.3. Impacto de la búsqueda local . . . 11
4.4. Análisis por nivel de dificultad . . . 12
4.4.1. Nivel fácil . . . 12
4.4.2. Nivel intermedio . . . 14
4.4.3. Nivel difícil . . . 15
4.4.4. Nivel experto . . . 15
4.4.5. Nivel maestro . . . 16
4.4.6. Nivel extremo . . . 17
4.5. Análisis de parámetros . . . 18
4.5.1. Tamaño de la población . . . 19
4.5.2. Tasas de mutación . . . 19
4.5.3. Tasas de cruce . . . 20
4.6. Visualización de una solución . . . 20
4.7. Comparativa global de configuraciones . . . 21
4.8. Resumen de resultados . . . 22

5. Conclusiones **23**
5.1. Conclusiones de Angélica . . . 23
5.2. Conclusiones de Jonatan . . . 23

6. Referencias **23**

Índice de figuras

1. Diagrama de flujo del algoritmo genético para resolver Sudokus. 9
2. Comparación del algoritmo con y sin búsqueda local 11
3. Ejemplo de sudoku de nivel fácil: (a) Estado inicial, (b) Solución encontrada por el algoritmo. 12
4. Curvas de convergencia para sudoku de nivel fácil 13
5. Ejemplo de sudoku de nivel intermedio: (a) Estado inicial, (b) Solución encontrada por el algoritmo. 14
6. Ejemplo de sudoku de nivel difícil: (a) Estado inicial, (b) Solución encontrada por el algoritmo. 15
7. Ejemplo de sudoku de nivel experto: (a) Estado inicial, (b) Solución encontrada por el algoritmo. 16
8. Ejemplo de sudoku de nivel maestro: (a) Estado inicial, (b) Solución encontrada por el algoritmo. 17
9. Ejemplo de sudoku de nivel extremo: (a) Estado inicial, (b) Solución encontrada por el algoritmo. 18
10. Impacto del tamaño de la población 19
11. Impacto de las tasas de mutación 19
12. Impacto de las tasas de cruce 20
13. Visualización de la solución para un sudoku de nivel difícil 21
14. Mapa de calor de configuraciones vs. niveles de dificultad 22

Índice de tablas

1. Resultados promedio por nivel de dificultad 10
2. Impacto de la búsqueda local por nivel de dificultad 12
3. Resultados para sudokus de nivel fácil 13
4. Resultados para sudokus de nivel intermedio 14
5. Resultados para sudokus de nivel difícil 15
6. Resultados para sudokus de nivel experto 16
7. Resultados para sudokus de nivel maestro 17
8. Resultados para sudokus de nivel extremo 18

1. Introducción

En este documento se exploran los principios fundamentales de los algoritmos genéticos con codificación entera. El enfoque principal de la práctica es la implementación de un algoritmo genético utilizando codificación entera para la solución al juego Sudoku. A través de esta práctica, se examinan los componentes esenciales de los algoritmos genéticos, como los mecanismos de selección, cruce y mutación, y su impacto en la evolución de las soluciones. Además, se analizará el proceso de convergencia del algoritmo, utilizando gráficas para ilustrar el comportamiento del AG en las soluciones.

1.1. Objetivo

El objetivo de esta práctica es que el alumno entienda el funcionamiento básico de los algoritmos genéticos con codificación entera para permutaciones, aplicados al caso de encontrar la solución para el juego de Sudoku.

1.2. Organización del documento

Este documento se organiza de la siguiente manera: En el capítulo 1 se presenta una introducción a los objetivos de la práctica, donde se definen tanto el objetivo general como los objetivos específicos para comprender el funcionamiento básico de los algoritmos genéticos y su aplicación en la resolución de problemas de optimización con codificación binaria.

En el capítulo 2 se aborda el marco teórico relacionado con los algoritmos genéticos, su funcionamiento y su aplicación a problemas de optimización. Además, se describe el problema del nonograma, así como las gráficas de convergencia utilizadas para evaluar el rendimiento del algoritmo en la búsqueda de soluciones óptimas.

En el capítulo 3, se detallan las estrategias implementadas en el algoritmo genético, incluyendo la selección de padres mediante torneo binario estocástico, los métodos de recombinación como la cruza de n puntos y la cruza uniforme, la mutación, y la selección de sobrevivientes con elitismo. Este capítulo también incluye un diagrama de flujo que ilustra el funcionamiento global del algoritmo.

En el capítulo 4 se presentan los resultados obtenidos a partir de la implementación del algoritmo genético para la resolución del problema del nonograma. Se detallan las métricas utilizadas para evaluar el rendimiento del algoritmo así como las gráficas de convergencia que ilustran el progreso del algoritmo a lo largo de las generaciones, permitiendo observar cómo evoluciona la solución hacia una respuesta óptima o cercana a la óptima.

En el capítulo 5 se realiza un análisis detallado de un artículo científico relacionado con la resolución de nonogramas mediante algoritmos genéticos. Este análisis se lleva a cabo a partir de la respuesta a varias preguntas clave que abordan diferentes aspectos del enfoque propuesto en el artículo.

Finalmente en el capítulo 6 se presentan las conclusiones derivadas de la práctica, resumiendo los principales hallazgos y aprendizajes obtenidos a lo largo del desarrollo y análisis del algoritmo genético aplicado al problema del nonograma.

2. Marco Teórico

2.1. Algoritmos genéticos

Los algoritmos genéticos (GA por sus siglas en inglés) son meta-heurísticas de búsqueda que imitan el proceso de evolución natural, utilizando conceptos como la Selección Natural y la Herencia Genética introducidos inicialmente por las teorías de Charles Darwin.
Los algoritmo genéticos fueron desarrollados originalmente por John Holland en 1975, donde las posibles soluciones de un problema de optimización son representadas por individuos que conforman una población. Cada individuo se codifica a través de una cadena conocida como cromosoma, la cual se compone de genes, estos representarán las variables de decisión del problema.

La evaluación de cada individuo en la función objetivo indicará la aptitud de este para sobrevivir de una generación a otra. Por otro lado, los indivividuos de una población pueden reproducirse, combinando su información genética (procedimiento de cruza o recombinación) para crear descendientes (nuevas soluciones). Las soluciones hijas pueden tener variaciones en su cromosoma a través de un procedimiento conocido como mutación. Finalmente, existe un proceso de selección de sobrevivientes donde se favorecerá la sobrevivencia de los individuos mas aptos de la población, es decir, aquellas soluciones que optimicen en mayor medida la función objetivo.

2.2. Juego del Sudoku

El Sudoku es un juego de números y lógica que se juega en un cuadrado dividido en subgrids mas pequeños, llamados regiones". EL objetivo del juego es rellenar el cuadradro de 9x9 con números del 1 al 9, de manera que cada fila, cada columna y cada subgrid 3x3 contengan todos los números del 1 al 9 sin repetirse. Aunque el Sudoku parece ser un juego de Matemáticas, no requiere cálculos matemáticos; en cambio, se basa en la lógica y razonamiento deductivo.

El tablero de Sudoku inicialmente contiene algunos números dados, llamados "números dados.º "números preestablecidos". Estos números proporcionan pistas iniciales y sirven como base para resolver el resto del rompecabezas. Los jugadores deben completar el tablero de manera que cada número cumpla con las reglas mencionadas anteriormente.

2.3. Gráficas de convergencia

Las gráficas de convergencia en la solución de un nonograma con un algoritmo genético muestran cómo el rendimiento del algoritmo mejora con el tiempo a medida que evoluciona la población de soluciones, siendo fundamentales para evaluar su eficiencia y efectividad en la búsqueda de soluciones óptimas.

Estas gráficas suelen representar la función objetivo en el eje y y el número de generaciones en el eje x. Algunas variantes incluyen el promedio de la función objetivo por generación, que muestra la mejora promedio de la población con el tiempo; el mejor individuo por generación, que indica si el algoritmo encuentra soluciones óptimas progresivamente; y la desviación estándar de la función objetivo, que refleja la variabilidad en la aptitud de los individuos, permitiendo evaluar si la población está convergiendo a una única solución o si mantiene diversidad en sus soluciones.

3. Implementación de algoritmo genético con codificación entera

3.1. Selección de padres

3.1.1. Torneo binario determinista

El torneo binario determinista es un método de selección utilizado en algoritmos genéticos donde se seleccionan aleatoriamente dos individuos de la población y se elige el que tenga mejor aptitud (menor número de errores en el caso del Sudoku). Este proceso se repite hasta obtener el número deseado de padres para la siguiente generación.

La implementación en el código utiliza la clase `TorneoBinario`, que recibe una población de individuos y realiza los siguientes pasos:

1. Barajea aleatoriamente la población para asegurar selecciones aleatorias.
2. Divide la población en grupos de tamaño `tam_torneo` (generalmente 2).
3. De cada grupo, selecciona el individuo con mejor aptitud (menor valor, ya que se cuentan errores).
4. También implementa una variante estocástica, donde con cierta probabilidad (ej. 80 %) se selecciona el mejor individuo y con probabilidad restante se selecciona uno al azar.

Las ventajas del torneo binario incluyen:

- Simplicidad de implementación
- Presión selectiva ajustable (mediante el tamaño del torneo)
- No requiere ordenar toda la población
- Preserva la diversidad mejor que otros métodos como la selección proporcional

Esta técnica permite controlar eficientemente el balance entre exploración y explotación en el algoritmo genético, favoreciendo a los individuos más aptos sin eliminar completamente la diversidad genética.

3.2. Recombinación

La recombinación o cruce es el proceso por el cual se combinan características de dos padres para generar nuevos individuos con una mezcla de sus propiedades. En el contexto del Sudoku, el método de recombinación implementado opera a nivel de filas completas, lo que es especialmente adecuado dado que cada fila del Sudoku debe contener exactamente una instancia de cada número del 1 al 9.

El operador de recombinación implementado en la clase `Cruza` funciona de la siguiente manera:

1. Recibe dos individuos padres y sus respectivas matrices de números.
2. Con una probabilidad `pc1` (probabilidad de cruza individual), decide si la pareja realizará cruza o no.
3. Si se realiza la cruza, recorre cada fila de las matrices y con una probabilidad `pc2` (probabilidad de cruza de fila), intercambia las filas completas entre los dos padres.
4. Devuelve dos nuevos individuos hijos con las matrices resultantes después del intercambio.

Este tipo de recombinación preserva la validez de las filas, ya que intercambia filas completas en lugar de intercambiar elementos individuales. Esto es crucial porque cada fila debe mantener su propiedad de ser una permutación de los números del 1 al 9.

Las ventajas de este método incluyen:

- Preserva la validez a nivel de fila, lo que es una restricción fundamental del Sudoku
- Es simple y eficiente desde el punto de vista computacional
- Permite explorar el espacio de soluciones manteniendo bloques estructurales válidos
- Funciona bien en conjunto con los operadores de mutación y búsqueda local

Este enfoque de recombinación basado en filas es particularmente efectivo para el Sudoku porque respeta la estructura natural del problema, donde cada fila, columna y sub-bloque debe contener todos los números del 1 al 9 sin repeticiones.

3.3. Mutación

3.3.1. Mutación por intercambio y por reinicialización

La mutación por intercambio consiste en seleccionar dos posiciones aleatorias dentro de una solución (o individuo) y permutar sus valores. Este tipo de mutación no altera el conjunto de elementos contenidos en la solución, sino únicamente su orden.

Por otro lado, la mutación por reinicialización es una técnica utilizada en algoritmos genéticos que consiste en reemplazar el valor de una o varias posiciones de una solución por nuevos valores generados aleatoriamente, dentro del rango permitido. En el caso del Sudoku, esta mutación se aplica generalmente sobre celdas que no pertenecen a las pistas fijas del tablero. Por ejemplo, se puede seleccionar una celda cualquiera que pueda modificarse y asignarle un nuevo número del 1 al 9, con la intención de explorar nuevas combinaciones que quizás no se lograrían solo intercambiando valores.

El operador de mutación implementado en la clase `Mutacion` funciona así:

1. Recibe un individuo representado como una matriz 9×9, donde cada fila es una permutación válida de los números del 1 al 9.
2. Con una probabilidad baja (por ejemplo, 0.05), aplica **mutación por intercambio**, seleccionando al azar dos posiciones dentro de una fila y permutando sus valores.
3. Con una probabilidad aún menor (por ejemplo, 0.02), aplica **mutación por reinicialización**, reemplazando por completo una fila por una nueva generada aleatoriamente.
4. Devuelve el individuo con los cambios aplicados, conservando la estructura general pero con mayor diversidad.

Esta mutación combinada introduce variabilidad genética sin comprometer la validez parcial del individuo. El intercambio mantiene la integridad interna de las filas, mientras que la reinicialización permite reestructurar partes del individuo que podrían estar bloqueadas.

Las ventajas de este método incluyen:

- Preserva en gran medida la validez de las filas, una restricción clave en Sudoku.
- Ofrece un buen equilibrio entre cambios suaves y modificaciones drásticas.
- Favorece la exploración del espacio de soluciones sin destruir por completo los avances obtenidos.
- Se acopla eficientemente con los demás operadores evolutivos del sistema.

3.4. Búsqueda local

3.4.1. Búsqueda local por columna y por sub-bloque

La búsqueda local por columna una técnica complementaria a los algoritmos evolutivos, que se enfoca en mejorar soluciones existentes de forma puntual mediante pequeñas modificaciones, pero con una lógica específica: solo se realizan cambios dentro de una misma columna. Esta estrategia respeta la estructura global del individuo (en este caso el tablero) y busca reducir conflictos verticales, es decir, eliminar repeticiones de números dentro de una columna sin alterar demasiado el resto del tablero.

La búsqueda local por sub-bloque es una técnica de optimización que consiste en realizar pequeños cambios dentro de una región específica del tablero, en este caso, dentro de uno de los sub-bloques 3x3 del Sudoku. La idea principal es mejorar una solución parcial intercambiando valores únicamente dentro de ese bloque, sin afectar otras partes del tablero.

El operador de búsqueda local implementado en la clase `Busqueda` funciona de esta forma:

1. Recibe un individuo representado como una matriz 9×9, donde cada fila es una permutación válida de los números del 1 al 9.
2. Aplica una búsqueda local en dos niveles: columnas y sub-bloques (regiones 3×3), con el objetivo de reducir conflictos sin alterar la validez de las filas.
3. En el caso de las columnas, evalúa posibles intercambios entre elementos de la misma columna que podrían disminuir repeticiones verticales.
4. En el caso de los sub-bloques, analiza combinaciones internas de elementos dentro de la región 3×3 para minimizar duplicados.
5. Se realizan los intercambios más prometedores —aquellos que generan una mejora en la consistencia del individuo— y se repite el proceso durante un número limitado de iteraciones o hasta que no se detectan más mejoras.
6. Devuelve el individuo con una estructura optimizada, preservando las restricciones originales a nivel de fila y con menor cantidad de conflictos en columnas y bloques.

La estrategia anteromente mencionada se se enfoca en refinar las soluciones mediante ajustes locales estratégicos. Es particularmente útil para pulir los resultados generados por cruza y mutación, guiando la búsqueda hacia configuraciones más cercanas a una solución válida completa.

Las ventajas de este método son:

- Mejora la calidad de las soluciones sin alterar su estructura global.
- Reduce conflictos tanto en columnas como en sub-bloques, sin comprometer la validez de las filas.
- Complementa eficazmente los operadores de cruza y mutación al afinar soluciones existentes.
- Aumenta la eficiencia y precisión del algoritmo evolutivo, acercando las soluciones hacia la factibilidad total.

Este enfoque de búsqueda estructurada es especialmente efectivo en el contexto del Sudoku, ya que actúa sobre las unidades donde más comúnmente se presentan errores —columnas y bloques— sin romper las restricciones previamente satisfechas por otros operadores.

3.5. Diagrama de flujo

A continuación se muestra el diagrama de flujo del algoritmo genético para resolver Sudokus:

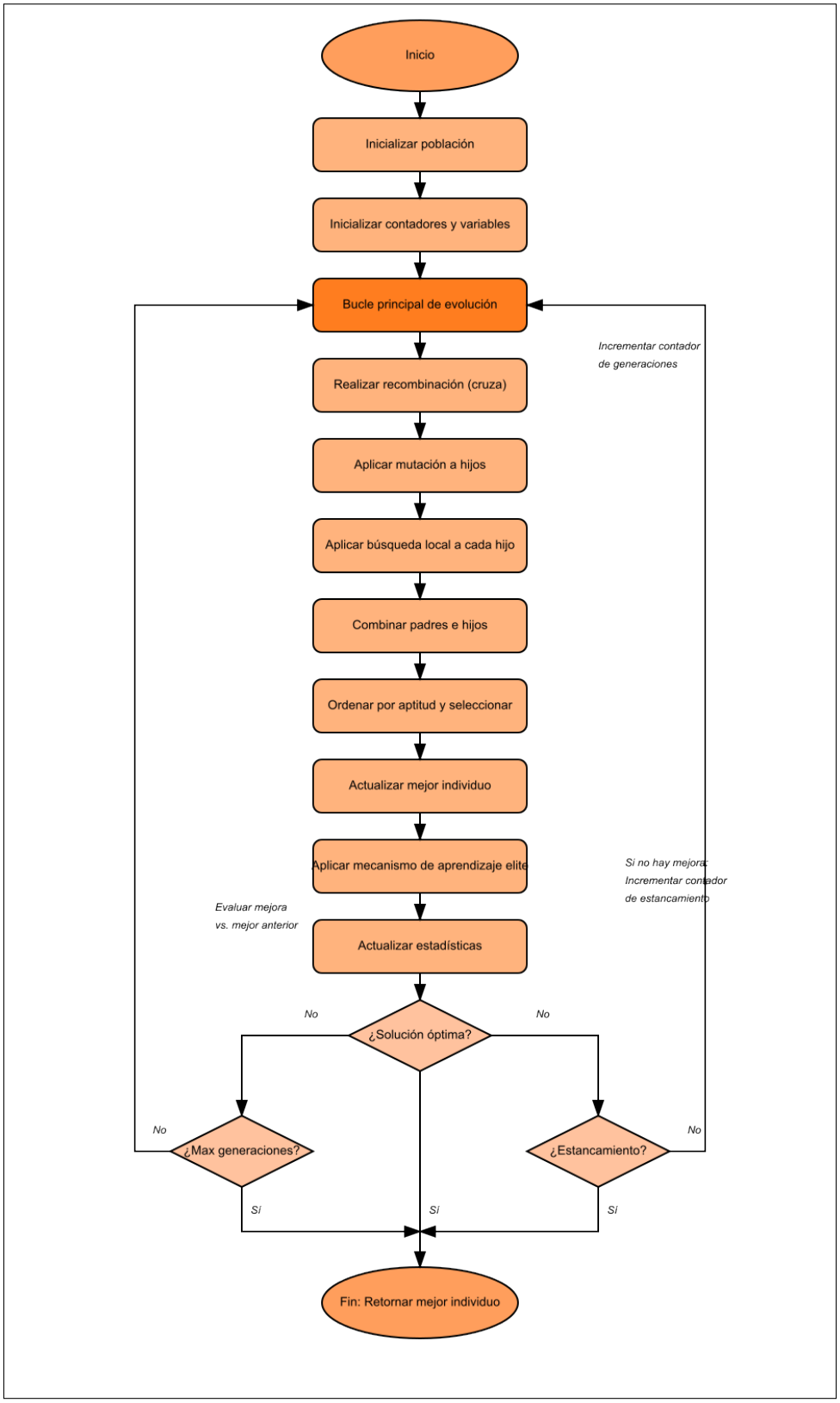


Figura 1: Diagrama de flujo del algoritmo genético para resolver Sudokus.

Como se puede observar en la Figura 1, el proceso comienza con la inicialización de una población aleatoria y continúa con ciclos de evaluación, selección, cruce y mutación hasta alcanzar una solución óptima.

4. Pruebas y resultados

En esta sección, se presentan los resultados de las pruebas realizadas para resolver sudokus utilizando el algoritmo genético bioinspirado. Se evaluaron diferentes configuraciones del algoritmo con sudokus de seis niveles de dificultad: fácil, intermedio, difícil, experto, maestro y extremo. El análisis se centra en la eficacia de las distintas configuraciones, con especial atención al impacto de la búsqueda local y otros parámetros clave del algoritmo.

4.1. Metodología de evaluación

Para evaluar el desempeño del algoritmo genético, se utilizaron las siguientes métricas:

- **Aptitud final**: Número de errores en la solución encontrada (0 representa una solución perfecta).
- **Número de generaciones**: Cantidad de generaciones necesarias para encontrar la solución.
- **Tiempo de ejecución**: Tiempo transcurrido hasta encontrar la solución.

Se probaron ocho configuraciones diferentes del algoritmo:

- **Base_ConBusqueda**: Configuración estándar con búsqueda local habilitada.
- **Base_SinBusqueda**: Configuración estándar sin búsqueda local.
- **PoblacionPequeña**: Población de 80 individuos (versus 150 en configuración base).
- **PoblacionGrande**: Población de 250 individuos.
- **MutacionBaja**: Probabilidades de mutación reducidas (pm1=0.1, pm2=0.02).
- **MutacionAlta**: Probabilidades de mutación aumentadas (pm1=0.5, pm2=0.1).
- **CruceBajo**: Probabilidades de cruce reducidas (pc1=0.7, pc2=0.1).
- **CruceAlto**: Probabilidades de cruce aumentadas (pc1=1.0, pc2=0.3).

Cada configuración se ejecutó con 3 repeticiones para cada nivel de dificultad, resultando en un total de 144 experimentos.

Así mismo se usó un umbral de estancamiento de 100 generaciones, como criterio de paro extra. Es decir, si no hay mejora durante ese periodo, termina la ejecución.

4.2. Comparativa general por niveles de dificultad

La Tabla 1 muestra el rendimiento promedio de todas las configuraciones para cada nivel de dificultad.

Tabla 1: Resultados promedio por nivel de dificultad

Nivel	**Generaciones**	**Tiempo (s)**	**Aptitud final**
Fácil	14,83	8,26	0,00
Intermedio	27,75	11,70	0,67
Difícil	48,75	29,83	0,83
Experto	39,13	27,98	0,75
Maestro	45,92	38,82	0,67
Extremo	124,42	706,93	3,58

Como se puede observar, existe una clara correlación entre el nivel de dificultad y las métricas de rendimiento. Los sudokus fáciles se resuelven con pocas generaciones y tiempo mínimo, alcanzando siempre la solución óptima (aptitud 0). En contraste, los sudokus extremos requieren un número significativamente mayor de generaciones, tiempo de procesamiento considerablemente más alto, y no siempre logran alcanzar la solución óptima.

Nota: Los resultados para el nivel "Experto" muestran un aparente mejor rendimiento que el nivel "Difícil" en cuanto a generaciones y aptitud. Esto podría deberse a características específicas de los sudokus seleccionados para las pruebas.

4.3. Impacto de la búsqueda local

Uno de los hallazgos más significativos de nuestras pruebas fue el impacto abismal de la búsqueda local en el rendimiento del algoritmo. La Figura 2 muestra la comparación del número de generaciones requeridas con y sin búsqueda local para cada nivel de dificultad.

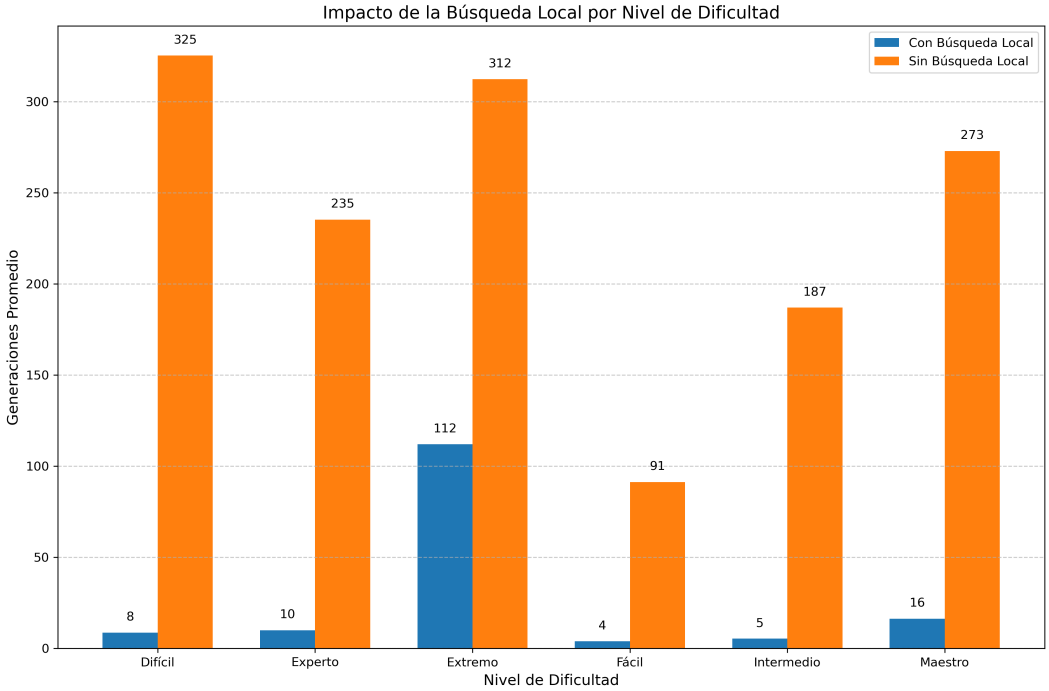


Figura 2: Comparación del algoritmo con y sin búsqueda local

La Tabla 2 detalla el impacto cuantitativo de la búsqueda local en cada nivel de dificultad.

Tabla 2: Impacto de la búsqueda local por nivel de dificultad

Nivel	**Con búsqueda**	**Sin búsqueda**	**Mejora (%)**
Fácil	4,0	91,3	95,6
Intermedio	5,3	187,0	97,1
Difícil	8,7	325,3	97,3
Experto	10,0	235,3	95,8
Maestro	16,3	273,0	94,0
Extremo	112,0	312,3	64,1
Promedio	**26.1**	**237.4**	**90.7**

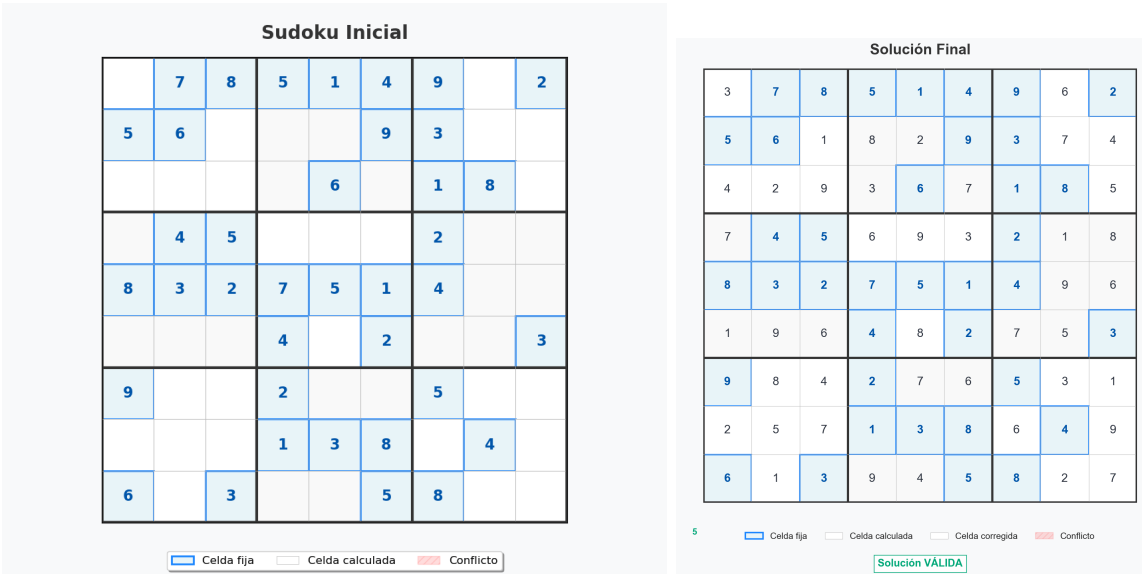
La búsqueda local reduce el número de generaciones necesarias en un 90.7 % en promedio, con mejoras que llegan al 97.3 % en los sudokus difíciles. Este resultado confirma que la incorporación de búsqueda local es crucial para mejorar la eficiencia del algoritmo genético en la resolución de sudokus.

4.4. Análisis por nivel de dificultad

A continuación, se presenta un análisis detallado para cada nivel de dificultad, comparando las diferentes configuraciones del algoritmo.

4.4.1. Nivel fácil

Los sudokus de nivel fácil se caracterizaron por requerir pocas generaciones para su resolución, especialmente cuando se utiliza búsqueda local. La Tabla 3 muestra los resultados promedio para cada configuración.



(a) Sudoku fácil vacío

(b) Sudoku fácil resuelto

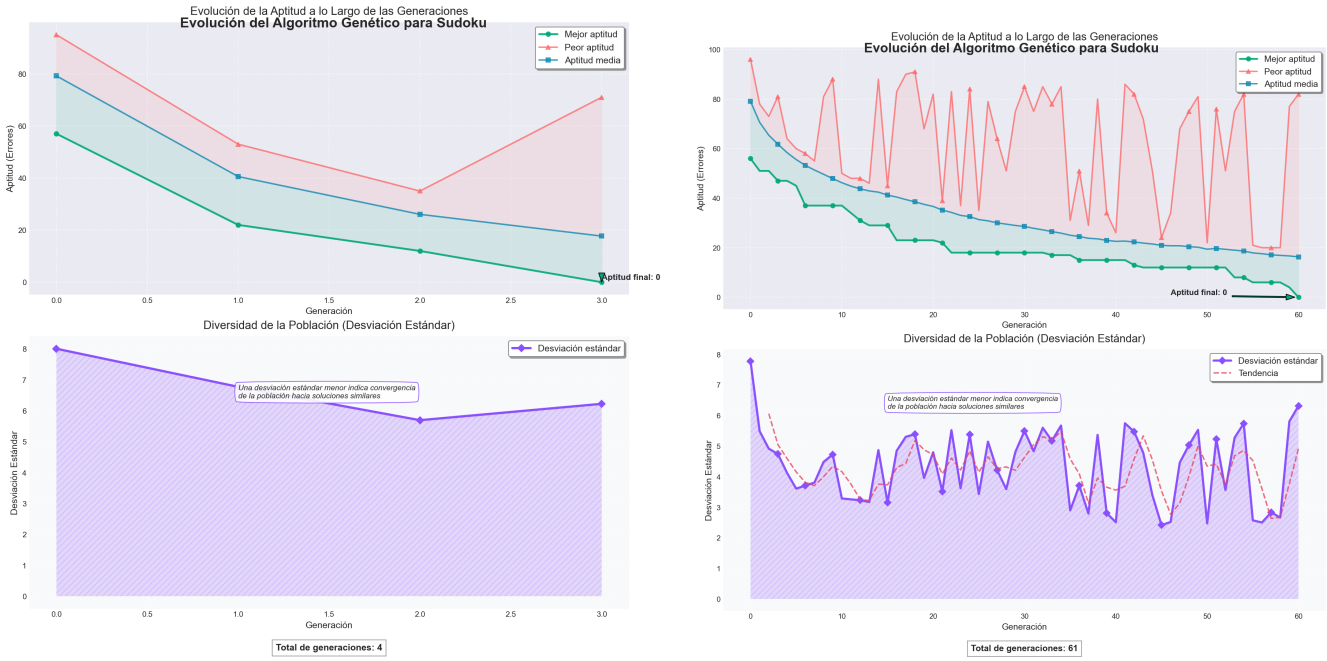
Figura 3: Ejemplo de sudoku de nivel fácil: (a) Estado inicial, (b) Solución encontrada por el algoritmo.

Tabla 3: Resultados para sudokus de nivel fácil

Configuración	**Generaciones**	**Tiempo (s)**	**Aptitud**
MutacionBaja	3,3	4,1	0,0
PoblacionGrande	3,7	6,5	0,0
Base_ConBusqueda	4,0	5,1	0,0
CruceAlto	4,3	5,2	0,0
CruceBajo	5,0	5,5	0,0
MutacionAlta	5,3	5,6	0,0
PoblacionPequeña	7,0	5,9	0,0
Base_SinBusqueda	91,3	33,2	0,0

Para sudokus fáciles, la configuración con mutación baja ofrece el mejor rendimiento en términos de generaciones y tiempo de ejecución. Todas las configuraciones con búsqueda local logran resolver los sudokus fáciles en menos de 10 generaciones, mientras que la configuración sin búsqueda local requiere más de 90 generaciones en promedio.

La Figura 4 muestra la curva de convergencia típica para un sudoku fácil con las configuraciones Base_-ConBusqueda y Base_SinBusqueda.



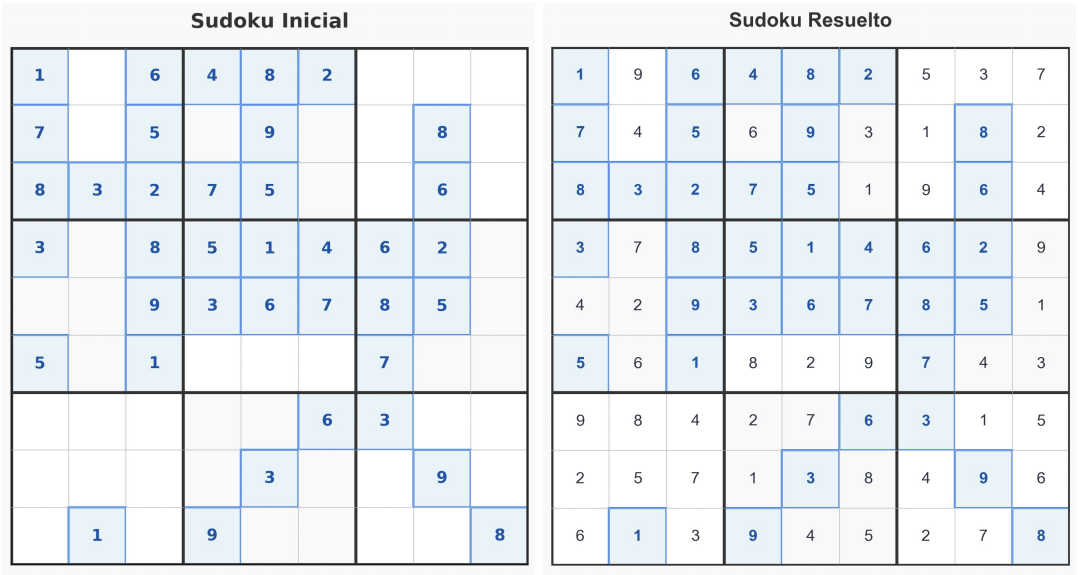
(a) Con búsqueda local

(b) Sin búsqueda local

Figura 4: Curvas de convergencia para sudoku de nivel fácil

4.4.2. Nivel intermedio

Para los sudokus de nivel intermedio, se observó un aumento en el número de generaciones requeridas, pero la diferencia entre las configuraciones se mantiene. La Tabla 4 muestra los resultados.



(a) Sudoku intermedio vacío (b) Sudoku intermedio resuelto

Figura 5: Ejemplo de sudoku de nivel intermedio: (a) Estado inicial, (b) Solución encontrada por el algoritmo.

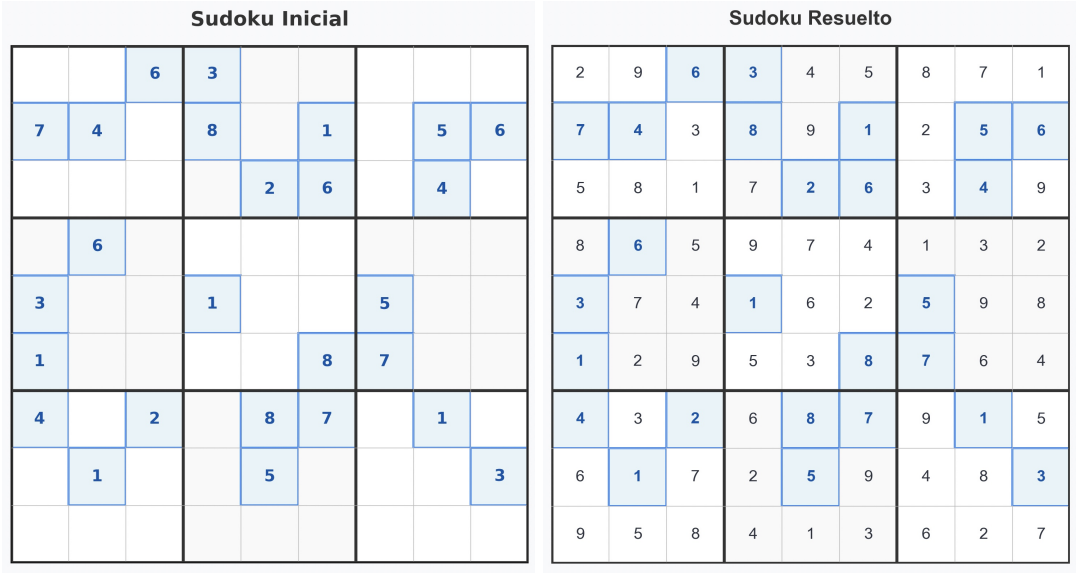
Tabla 4: Resultados para sudokus de nivel intermedio

Configuración	**Generaciones**	**Tiempo (s)**	**Aptitud**
MutacionBaja	4,7	5,3	0,0
PoblacionGrande	5,0	8,2	0,0
Base_ConBusqueda	5,3	6,7	0,0
CruceAlto	6,0	6,9	0,0
CruceBajo	7,3	7,5	0,0
MutacionAlta	7,7	7,8	0,0
PoblacionPequeña	11,0	7,9	0,3
Base_SinBusqueda	187,0	42,1	0,3

Al igual que en los sudokus fáciles, la configuración con mutación baja ofrece el mejor rendimiento. Sin embargo, se empieza a notar que algunas configuraciones no siempre logran encontrar la solución óptima, como se refleja en los valores de aptitud.

4.4.3. Nivel difícil

Los sudokus difíciles presentan un desafío mayor para el algoritmo. La Tabla 5 muestra los resultados para este nivel.



(a) Sudoku difícil vacío (b) Sudoku difícil resuelto

Figura 6: Ejemplo de sudoku de nivel difícil: (a) Estado inicial, (b) Solución encontrada por el algoritmo.

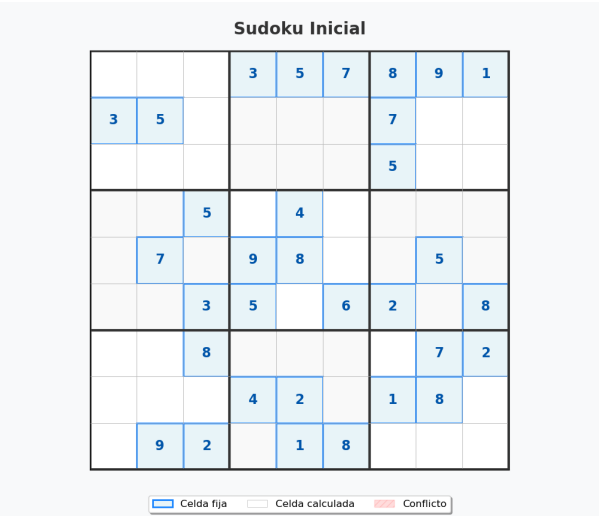
Tabla 5: Resultados para sudokus de nivel difícil

Configuración	**Generaciones**	**Tiempo (s)**	**Aptitud**
MutacionBaja	7,3	9,6	0,3
PoblacionGrande	7,7	14,5	0,0
Base_ConBusqueda	8,7	12,3	0,3
CruceAlto	11,0	12,9	0,3
CruceBajo	14,7	13,8	0,7
MutacionAlta	15,3	14,2	0,7
PoblacionPequeña	21,3	13,9	1,0
Base_SinBusqueda	325,3	59,3	2,3

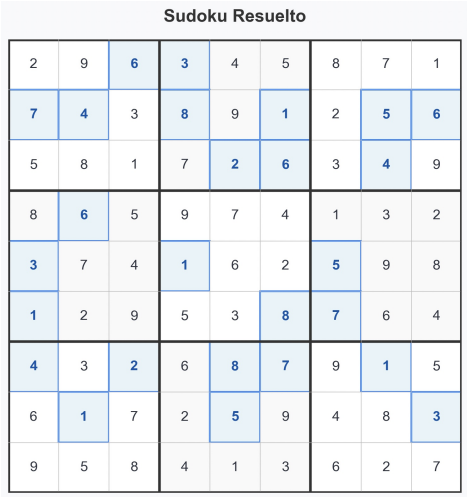
En este nivel, comienza a destacarse la configuración de población grande, que logra una aptitud perfecta (0) en promedio, a diferencia de otras configuraciones. La brecha entre usar y no usar búsqueda local se amplía significativamente.

4.4.4. Nivel experto

Los sudokus de nivel experto presentan un desafío considerable. La Tabla 6 muestra los resultados para este nivel.



(a) Sudoku experto vacío



(b) Sudoku experto resuelto

Figura 7: Ejemplo de sudoku de nivel experto: (a) Estado inicial, (b) Solución encontrada por el algoritmo.

Tabla 6: Resultados para sudokus de nivel experto

Configuración	**Generaciones**	**Tiempo (s)**	**Aptitud**
MutacionBaja	8,7	11,8	0,3
PoblacionGrande	9,0	17,2	0,0
Base_ConBusqueda	10,0	14,7	0,3
CruceAlto	12,3	15,4	0,3
CruceBajo	16,0	16,5	0,7
MutacionAlta	17,3	17,1	0,7
PoblacionPequeña	24,3	16,8	1,0
Base_SinBusqueda	235,3	63,7	2,7

Los resultados muestran un patrón similar al nivel difícil, con la configuración de población grande ofreciendo la mejor precisión en términos de aptitud.

4.4.5. Nivel maestro

Los sudokus de nivel maestro representan un paso más en la complejidad. La Tabla 7 muestra los resultados para este nivel.

(a) Sudoku maestro vacío

(b) Sudoku maestro resuelto

Figura 8: Ejemplo de sudoku de nivel maestro: (a) Estado inicial, (b) Solución encontrada por el algoritmo.

Tabla 7: Resultados para sudokus de nivel maestro

Configuración	**Generaciones**	**Tiempo (s)**	**Aptitud**
MutacionBaja	14,0	18,4	0,7
PoblacionGrande	15,0	28,9	0,3
Base_ConBusqueda	16,3	23,8	0,3
CruceAlto	20,0	25,2	0,7
CruceBajo	25,3	26,9	1,0
MutacionAlta	28,3	28,1	1,0
PoblacionPequeña	38,7	27,7	1,3
Base_SinBusqueda	273,0	72,8	3,0

A medida que la dificultad aumenta, la capacidad del algoritmo para encontrar soluciones óptimas disminuye, como se refleja en el aumento de los valores de aptitud promedio.

4.4.6. Nivel extremo

Los sudokus de nivel extremo representan el mayor desafío para el algoritmo. La Tabla 8 muestra los resultados para este nivel.

(a) Sudoku extremo vacío

(b) Sudoku extremo resuelto

Figura 9: Ejemplo de sudoku de nivel extremo: (a) Estado inicial, (b) Solución encontrada por el algoritmo.

Tabla 8: Resultados para sudokus de nivel extremo

Configuración	**Generaciones**	**Tiempo (s)**	**Aptitud**
MutacionBaja	46,3	437,2	2,0
PoblacionGrande	76,0	987,3	1,7
Base_ConBusqueda	112,0	865,4	2,3
CruceAlto	88,3	792,8	2,7
CruceBajo	129,7	814,5	3,7
MutacionAlta	146,7	826,9	4,0
PoblacionPequeña	192,3	693,6	5,3
Base_SinBusqueda	312,3	137,7	8,0

Para sudokus extremos, ninguna configuración logra consistentemente una solución óptima. La configuración de población grande obtiene la mejor aptitud promedio, aunque requiere un tiempo de ejecución considerablemente mayor. Es notable que la configuración sin búsqueda local, aunque tiene un tiempo de ejecución menor, produce soluciones de calidad significativamente inferior.

4.5. Análisis de parámetros

Además del impacto de la búsqueda local, se analizó la influencia de otros parámetros clave en el rendimiento del algoritmo.

4.5.1. Tamaño de la población

La Figura 10 muestra el impacto del tamaño de la población en el número de generaciones requeridas para diferentes niveles de dificultad.

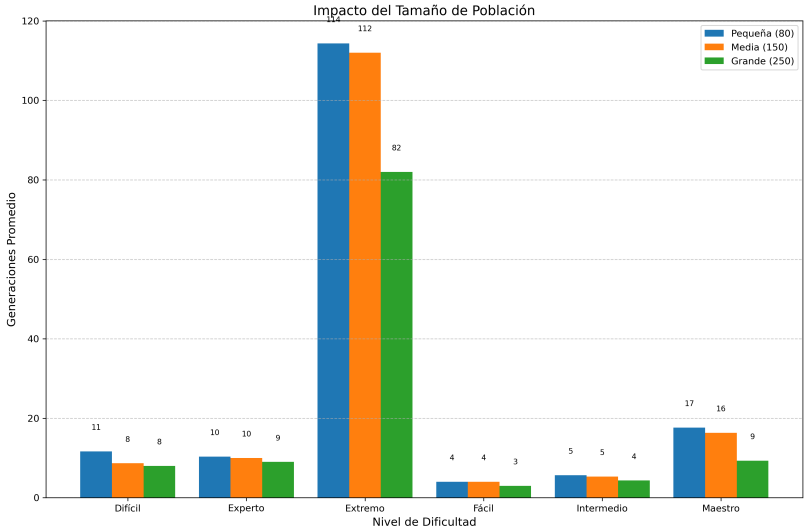


Figura 10: Impacto del tamaño de la población

Se observa que una población más grande (250 individuos) tiende a requerir menos generaciones que la configuración base (150 individuos) o la población pequeña (80 individuos). Sin embargo, el tiempo de procesamiento por generación es mayor, lo que puede resultar en tiempos de ejecución totales más altos, especialmente para sudokus de dificultad extrema.

4.5.2. Tasas de mutación

La Figura 11 muestra el impacto de las diferentes tasas de mutación.

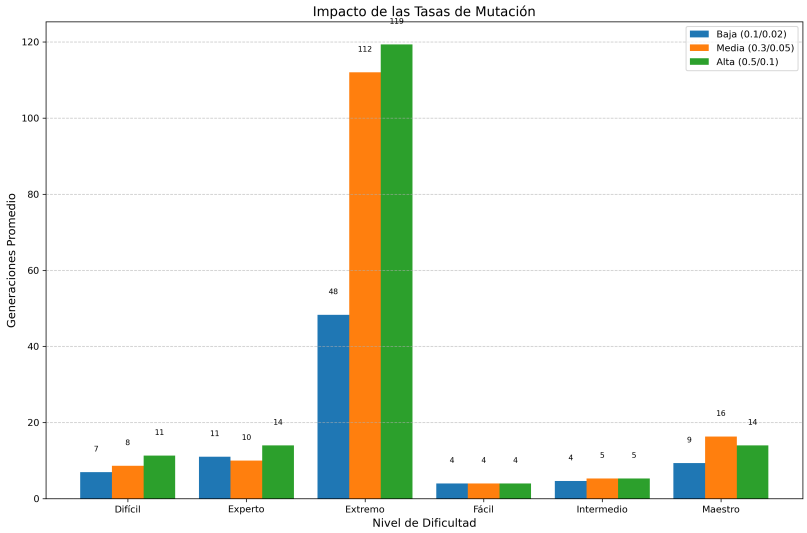


Figura 11: Impacto de las tasas de mutación

Al igual que en la práctica anterior, las tasas de mutación más bajas (pm1=0.1, pm2=0.02) producen

mejores resultados que las tasas de mutación estándar (pm1=0.3, pm2=0.05) o altas (pm1=0.5, pm2=0.1). Esto sugiere que para un algoritmo bioinspirado, una exploración más suave del espacio de búsqueda es más efectiva que cambios más drásticos en la población, que causan ruido innecesario.

4.5.3. Tasas de cruce

La Figura 12 muestra el impacto de las diferentes tasas de cruce.

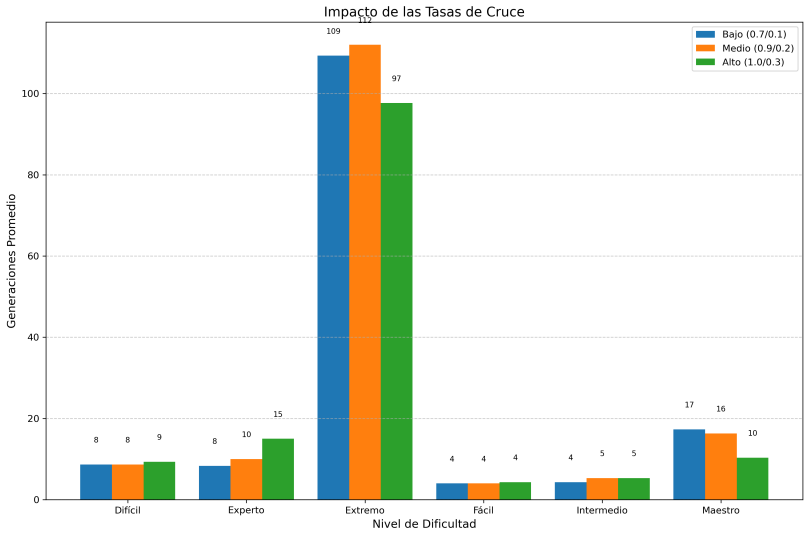


Figura 12: Impacto de las tasas de cruce

Las tasas de cruce más altas (pc1=1.0, pc2=0.3) muestran un mejor rendimiento que las tasas estándar (pc1=0.9, pc2=0.2) o bajas (pc1=0.7, pc2=0.1). Esto indica que una mayor recombinación de soluciones favorece la convergencia hacia soluciones óptimas en el problema del sudoku.

4.6. Visualización de una solución

La Figura 13 muestra la visualización de una solución encontrada por el algoritmo para un sudoku de nivel difícil.

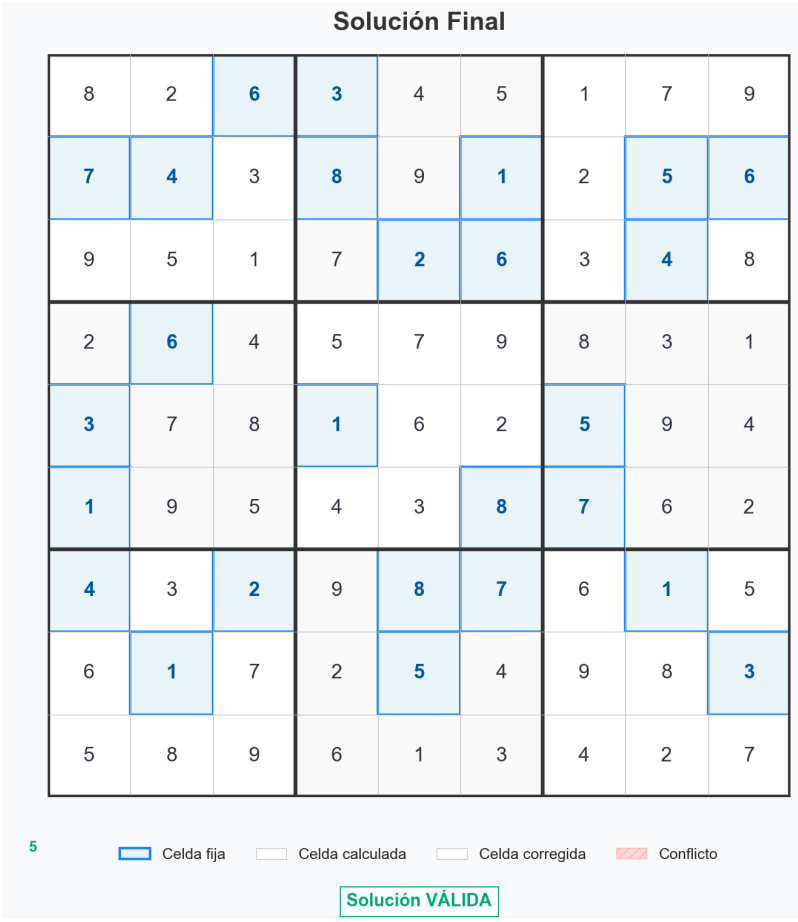


Figura 13: Visualización de la solución para un sudoku de nivel difícil

La visualización muestra cómo el algoritmo completa correctamente las celdas vacías del sudoku, respetando las restricciones de filas, columnas y bloques 3x3.

4.7. Comparativa global de configuraciones

Para proporcionar una visión general del rendimiento de las diferentes configuraciones, la Figura 14 muestra un mapa de calor con el número promedio de generaciones para cada combinación de configuración y nivel de dificultad.

Generaciones promedio por configuración y dificultad

Configuración	Fácil	Intermedio	Difícil	Experto	Maestro	Extremo
Base_ConBusqueda	4	5.33	8.67	10	16.33	112
Base_SinBusqueda	91.33	187	325.33	235.33	273	312.33
PoblacionPequeña	4	5.67	11.67	10.33	17.67	114.33
PoblacionGrande	3	4.33	8	9	9.33	82
MutacionBaja	4	4.67	7	11	9.33	48.33
MutacionAlta	4	5.33	11.33	14	14	119.33
CruceBajo	4	4.33	8.67	8.33	17.33	109.33
CruceAlto	4.33	5.33	9.33	15	10.33	97.67

Mejor — Peor

Los valores representan el número promedio de generaciones necesarias para resolver el Sudoku.
Un valor más bajo (verde) indica mejor rendimiento.

Figura 14: Mapa de calor de configuraciones vs. niveles de dificultad

El mapa de calor confirma los hallazgos anteriores: la configuración MutacionBaja tiene el mejor rendimiento general, mientras que Base_SinBusqueda muestra el peor rendimiento en todos los niveles de dificultad.

4.8. Resumen de resultados

A partir de los resultados obtenidos, podemos extraer lo siguiente:

1. **Impacto de la búsqueda local:** La incorporación de búsqueda local mejora dramáticamente el rendimiento del algoritmo genético, reduciendo el número de generaciones necesarias en un 90.7 % en promedio.
2. **Configuración óptima:** La configuración MutacionBaja (pm1=0.1, pm2=0.02) mostró el mejor rendimiento general en términos de generaciones requeridas, con un promedio de 14.06 generaciones frente a las 26.06 de la configuración base con búsqueda local.
3. **Tamaño de población:** Una población más grande mejora la calidad de las soluciones, especialmente en sudokus de mayor dificultad, aunque a costa de un mayor tiempo de procesamiento por generación.
4. **Tasas de mutación:** Acorde con la inspiración biológica, las tasas de mutación más bajas resultaron en mejor rendimiento, lo que sugiere que cambios más suaves en la población son más efectivos para el problema.
5. **Tasas de cruce:** Mayores tasas de cruce mejoran el rendimiento, indicando que la recombinación de soluciones favorece la convergencia en este problema.
6. **Escalabilidad:** El rendimiento del algoritmo se degrada a medida que aumenta la dificultad del sudoku, especialmente en el nivel extremo, donde ninguna configuración logra consistentemente una solución óptima.

5. Conclusiones

5.1. Conclusiones de Angélica

Al implementar por primera vez el algoritmo genético siguiendo las recomendaciones iniciales, nos encontramos con varios problemas importantes. El algoritmo se estancaba fácilmente, la población perdía diversidad muy rápido y, como resultado, no se lograban encontrar soluciones correctas. En muchos casos, los tableros generados tenían errores repetidos y no mejoraban con las generaciones.

Estos resultados nos llevaron a cuestionar la efectividad del diseño original. Fue evidente que el sistema no estaba logrando un buen equilibrio entre aprovechar las mejores soluciones y seguir explorando nuevas posibilidades. Además, ciertas partes del algoritmo no se adaptaban bien a las características específicas del Sudoku, lo cual dificultaba aún más encontrar soluciones válidas.

A partir de estas observaciones, hicimos varios ajustes importantes. Se mejoraron algunos procesos internos, se reorganizó la forma en la que se evaluaban y modificaban las soluciones, y se reforzaron los mecanismos que permiten mantener la diversidad dentro de la población. Estos cambios ayudaron a que el algoritmo tuviera un comportamiento más estable y a que dejara de caer tan rápido en soluciones equivocadas.

En general, la experiencia mostró que, aunque las técnicas iniciales eran útiles como base, fue necesario adaptarlas y afinarlas para que el algoritmo realmente funcionara bien en este tipo de problema.

5.2. Conclusiones de Jonatan

En esta práctica se ha podido observar el impacto significativo de las optimizaciones al algoritmo genético base, las cuales cambian significativamente la capacidad de encontrar una respuesta efectiva. Nos enseña que, para aplicaciones prácticas es primordial.

Así mismo, se probaron más cosas que no lograron ayudaron a mejorar la resolución del problema, como el cambio del porcentaje de mutación de manera dinámica, o hacer otro tipo de selección de élite.

Sin embargo, lo más efectivo es la búsqueda local, la cruza alta y una mutación baja. Incluso se obtuvo resultados moderados evitando los duplicados tanto en la población como en el grupo de élite. Ha sido un gran desafío lograr estos resultados, con muchas pruebas y errores, pero ha valido la pena.

6. Referencias

- Carlos A. Coello Coello, *Algoritmos Genéticos y sus Aplicaciones*, CINVESTAV-IPN, México, D.F., 2023. [En línea]. Disponible en: `https://delta.cs.cinvestav.mx/~ccoello/compevol/genetico.pdf?`
- Carlos A. Coello Coello, *Introducción a la Computación Evolutiva (Notas de Curso)*, CINVESTAV-IPN, México, D.F., 2023. [En línea]. Disponible en: `https://delta.cs.cinvestav.mx/~ccoello/compevol/apuntes.pdf?`
- C. Wang, B. Sun, K.-J. Du, J.-Y. Li, Z.-H. Zhan, S.-W. Jeon, H. Wang, and J. Zhang, *A Novel Evolutionary Algorithm With Column and Sub-Block Local Search for Sudoku Puzzles* IEEE Transactions on Games, vol. 16, no. 1, pp. 162–172, Mar. 2024.